STOCKHOLM SCHOOL OF ECONOMICS

4339 Asset Pricing and Investments

Assignment 3

Evaluating Mutual Fund Performance

Sharpe ratios, information ratios and factor-model alphas, 2002–2026

Florian Radlmeier · Julius Gremminger

October 2026

Submitted as Jupyter notebook and PDF. All computations are reproducible from the notebook.

Contents

Numerical exercise: evaluating four mutual funds **2**

1. Data and monthly excess returns **4**
Fund prices from Yahoo Finance 4
Risk-free rate and factors from the Kenneth French data library 4
Merge and compute excess returns 5

2. Sharpe ratio and Information ratio **7**
Interpretation (Part 2) 9

3. Is there evidence of manager skill? Factor regressions **11**
Alpha summary: FF3 vs. FFC4 13
Interpretation (Part 3) 13

Numerical exercise: evaluating four mutual funds

Ticker	Fund
TRBCX	T. Rowe Price Blue Chip Growth
GLCGX	Goldman Sachs Large Cap Growth Insights
STSEX	BlackRock Exchange Portfolio (BlackRock Shares)
DFSVX	DFA U.S. Small Cap Value Portfolio

Sample: monthly returns, January 2002 – July 2026 (last price date 2026-07-01). **Data:** Yahoo Finance (adjusted fund prices) and the Kenneth French data library (Fama–French 3 factors, momentum, risk-free rate).

```
import io
import re
import zipfile

import numpy as np
import pandas as pd
import requests
import statsmodels.api as sm
import yfinance as yf
from IPython.display import Markdown, display

FUNDS = {
    "TRBCX": "T. Rowe Price Blue Chip Growth",
    "GLCGX": "Goldman Sachs Large Cap Growth Insights",
    "STSEX": "BlackRock Exchange Portfolio",
    "DFSVX": "DFA U.S. Small Cap Value",
}
TICKERS = list(FUNDS)

PRICE_START = "2001-12-01"  # one month early, so January 2002 has a return
PRICE_END = "2026-07-02"    # yfinance's end date is exclusive -> last bar is 2026-07-01
SAMPLE_START, SAMPLE_END = "2002-01", "2026-07"

FRENCH_URL = "https://mba.tuck.dartmouth.edu/pages/faculty/ken.french/ftp/"
ANN = 12  # months per year, used to annualize


def show_table(table, fmt="{:.3f}", title=None):
    # Display a DataFrame as a Markdown table, so it renders cleanly in the notebook and in the PDF.
    # fmt: one format string for all columns, or a dict {column: format}. Strings are shown as is.
    def cell(value, column):
        if isinstance(value, str):
            text = value
        elif pd.isna(value):
            return ""
        else:
            text = (fmt[column] if isinstance(fmt, dict) else fmt).format(value)
        return text.replace("-", "\N{MINUS SIGN}").replace("*", r"\*")  # typographic minus, literal
    ↪ stars

    labels = [str(label) for label in table.index]
    headers = [str(column) for column in table.columns]
    rows = [[cell(value, column) for column, value in row.items()] for _, row in table.iterrows()]
    # pandoc sizes wide PDF tables by these dash counts; headers can wrap, so count their longest
    ↪ word
    widths = [max(3, *map(len, labels))]
    for k, header in enumerate(headers):
```

```
        widths.append(max(*(len(word) for word in header.split()), *(len(r[k]) for r in rows)))
    lines = [
        "| " + " | ".join(headers) + " |",
        "|" + "-" * widths[0] + "|" + "|".join("-" * (w - 1) + ":" for w in widths[1:]) + "|",
    ]
    lines += [f"| {label} | " + " | ".join(row) + " |" for label, row in zip(labels, rows)]
    if title:
        lines.insert(0, f"**{title}**\n")
    display(Markdown("\n".join(lines)))
```

1. Data and monthly excess returns

Fund prices from Yahoo Finance

We use **adjusted** closing prices (`auto_adjust=True`). Mutual funds pay out dividends and capital-gain distributions, which make the raw price drop. Adjusted prices add these payouts back, so `pct_change()` gives the **total return**.

Yahoo's monthly bar dated the 1st of month t holds the **closing price at the end of month t**. The December 2001 bar is therefore the starting value for the January 2002 return.

```
prices = yf.download(
    TICKERS,
    start=PRICE_START,
    end=PRICE_END,
    interval="1mo",
    auto_adjust=True,
    progress=False,
)["Close"][TICKERS]
prices.index = prices.index.to_period("M")

fund_ret = prices.pct_change().dropna()

print(f"Prices : {prices.index[0]} -> {prices.index[-1]}")
print(f"Returns: {fund_ret.index[0]} -> {fund_ret.index[-1]}  ({len(fund_ret)} months)")
show_table(fund_ret.head(), "{:.4f}", title="Monthly fund returns (decimals), first five months")
```

```
Prices : 2001-12 -> 2026-07
Returns: 2002-01 -> 2026-07  (295 months)
```

Monthly fund returns (decimals), first five months

	TRBCX	GLCGX	STSEX	DFSVX
2002-01	−0.0290	−0.0227	−0.0093	0.0154
2002-02	−0.0292	−0.0371	0.0053	−0.0016
2002-03	0.0472	0.0260	0.0292	0.0986
2002-04	−0.0619	−0.0620	−0.0563	0.0453
2002-05	−0.0168	−0.0251	−0.0196	−0.0315

Risk-free rate and factors from the Kenneth French data library

We download the zipped CSV files directly:

- `F-F_Research_Data_Factors`: `Mkt-RF`, `SMB`, `HML`, `RF`
- `F-F_Momentum_Factor`: `Mom` (the Carhart momentum factor; "FFC" = Fama–French–Carhart)

Each file has a **monthly** block (dates as `YYYYMM`) followed by an **annual** block. We keep only the monthly rows. The values are in **percent**, so we divide by 100.

```
MONTHLY_ROW = re.compile(r"^\s*\d{6}\s*,")  # a line that starts with YYYYMM


def load_french_monthly(zip_name):
    # Download one Kenneth French CSV zip and return its monthly table in decimals.
    response = requests.get(FRENCH_URL + zip_name, timeout=60)
    response.raise_for_status()
```

```
    with zipfile.ZipFile(io.BytesIO(response.content)) as z:
        lines = z.read(z.namelist()[0]).decode("latin-1").splitlines()

    first = next(i for i, line in enumerate(lines) if MONTHLY_ROW.match(line))
    columns = [c.strip() for c in lines[first - 1].split(",")[1:]]  # header sits right above

    rows = []
    for line in lines[first:]:
        if not MONTHLY_ROW.match(line):
            break  # the monthly block ends where the annual block starts
        date, *values = [v.strip() for v in line.split(",")]
        rows.append([date, *map(float, values)])

    table = pd.DataFrame(rows, columns=["date", *columns])
    table.index = pd.PeriodIndex(table.pop("date"), freq="M")
    return table / 100


ff3 = load_french_monthly("F-F_Research_Data_Factors_CSV.zip")  # Mkt-RF, SMB, HML, RF
mom = load_french_monthly("F-F_Momentum_Factor_CSV.zip")        # Mom
factors = ff3.join(mom, how="inner")
show_table(
    factors.loc[SAMPLE_START:SAMPLE_END].head(),
    "{:.4f}",
    title="Factor returns and risk-free rate (decimals), first five months of the sample",
)
```

Factor returns and risk-free rate (decimals), first five months of the sample

	Mkt-RF	SMB	HML	RF	Mom
2002-01	−0.0144	0.0117	0.0342	0.0014	0.0366
2002-02	−0.0230	−0.0108	0.0234	0.0013	0.0688
2002-03	0.0424	0.0409	0.0108	0.0013	−0.0166
2002-04	−0.0521	0.0582	0.0401	0.0015	0.0769
2002-05	−0.0136	−0.0327	0.0152	0.0014	0.0312

Merge and compute excess returns

$$r^e_{i,t} = r_{i,t} - r_{f,t}$$

The market's excess return is French's `Mkt-RF` (value-weighted CRSP market minus the T-bill rate).

```
data = fund_ret.join(factors, how="inner").loc[SAMPLE_START:SAMPLE_END]

expected_months = len(pd.period_range(SAMPLE_START, SAMPLE_END, freq="M"))
assert len(data) == expected_months, "missing months after merging"
assert not data.isna().any().any(), "missing values in the merged data"

excess = data[TICKERS].sub(data["RF"], axis=0)  # fund excess returns
mkt_excess = data["Mkt-RF"]                     # market excess return

print(f"Sample: {data.index[0]} -> {data.index[-1]}  ({len(data)} months)")
show_table(excess.head(), "{:.4f}", title="Monthly excess returns (decimals), first five months")
```

```
Sample: 2002-01 -> 2026-07  (295 months)
```

Monthly excess returns (decimals), first five months

	TRBCX	GLCGX	STSEX	DFSVX
2002-01	−0.0304	−0.0241	−0.0107	0.0140
2002-02	−0.0305	−0.0384	0.0040	−0.0029
2002-03	0.0459	0.0247	0.0279	0.0973
2002-04	−0.0634	−0.0635	−0.0578	0.0438
2002-05	−0.0182	−0.0265	−0.0210	−0.0329

2. Sharpe ratio and Information ratio

Sharpe ratio: excess return per unit of total risk

$$SR_i = \frac{E[r_i^e]}{\sigma(r_i^e)}$$

We report the information ratio relative to the market in two versions.

1. Information ratio as defined in the course:

$$IR_i = \frac{\alpha_i}{\sigma(e_i)}$$

To get α and σ(e), we run the single-index (CAPM) regression of each fund's monthly excess return on the market's monthly excess return (`Mkt-RF`) over all 295 months:

$$r_{i,t}^e = \alpha_i + \beta_i \, r_{M,t}^e + e_{i,t}$$

- **α** is the intercept of this regression (Jensen's alpha): the average monthly return that the fund's market exposure does not explain.
- **σ(e)** is the standard deviation of the regression residuals $e_{i,t}$: the fund's non-market (residual) risk.

This IR is the key quantity of the **Treynor–Black model**. Combining the market with an active portfolio in optimal proportions gives the Sharpe ratio $S^* = \sqrt{S_M^2 + IR^2}$, so the IR measures how much a fund can add to an investor who already holds the market.

2. Practitioner version. Practitioners often skip the regression and compute the IR directly from the fund's return relative to its benchmark. For each month we take the active return $r_{i,t} - r_{M,t}$, the fund's return minus the market's return. The risk-free rate cancels, so this is also the difference of the two excess returns. The IR is the average active return divided by its standard deviation, the *tracking error*:

$$\widetilde{IR}_i = \frac{E[r_i - r_M]}{\sigma(r_i - r_M)} = \frac{\alpha_i - (1-\beta_i)\,E[r_M^e]}{\sqrt{(1-\beta_i)^2\sigma_M^2 + \sigma^2(e_i)}}$$

This needs only the two return series, so it can be computed directly from the data without estimating anything. The second expression links it to version 1: the two coincide when $\beta = 1$. When $\beta \neq 1$, the practitioner version also includes the effect of taking more (or less) market risk than the index.

Everything is annualized: means and alphas × 12, standard deviations × √12, so all ratios get × √12.

```
def annualized_ratio(series):
    return series.mean() / series.std() * np.sqrt(ANN)


# Single-index (CAPM) regression of each fund on the market: alpha, beta and sigma(e)
capm_alpha, capm_beta, capm_resid_vol = {}, {}, {}
for fund in TICKERS:
    capm = sm.OLS(excess[fund], sm.add_constant(mkt_excess)).fit()
    capm_alpha[fund] = capm.params["const"]
    capm_beta[fund] = capm.params["Mkt-RF"]
    capm_resid_vol[fund] = np.sqrt(capm.mse_resid)
capm_alpha, capm_beta, capm_resid_vol = map(pd.Series, (capm_alpha, capm_beta, capm_resid_vol))
```

```
active = excess.sub(mkt_excess, axis=0)  # r_i − r_M, only needed for the practitioner version

sharpe_table = pd.DataFrame(
    {
        "Mean excess (%/yr)": excess.mean() * ANN * 100,
        "Volatility (%/yr)": excess.std() * np.sqrt(ANN) * 100,
        "Sharpe ratio": excess.apply(annualized_ratio),
    }
)
sharpe_table.loc["Market"] = [
    mkt_excess.mean() * ANN * 100,
    mkt_excess.std() * np.sqrt(ANN) * 100,
    annualized_ratio(mkt_excess),
]

ir_table = pd.DataFrame(
    {
        "Alpha (%/yr)": capm_alpha * ANN * 100,
        "Beta": capm_beta,
        "σ(e) (%/yr)": capm_resid_vol * np.sqrt(ANN) * 100,
        "Information ratio": capm_alpha / capm_resid_vol * np.sqrt(ANN),
    }
)

practitioner_table = pd.DataFrame(
    {
        "Active return (%/yr)": active.mean() * ANN * 100,
        "Tracking error (%/yr)": active.std() * np.sqrt(ANN) * 100,
        "Practitioner IR": active.apply(annualized_ratio),
    }
)

show_table(
    sharpe_table,
    {"Mean excess (%/yr)": "{:.2f}", "Volatility (%/yr)": "{:.2f}", "Sharpe ratio": "{:.3f}"},
    title="Sharpe ratios (annualized)",
)
show_table(
    ir_table,
    {column: "{:.2f}" if "%" in column else "{:.3f}" for column in ir_table.columns},
    title="Information ratio IR = α / σ(e) from the CAPM regression (annualized)",
)
show_table(
    practitioner_table,
    {column: "{:.2f}" if "%" in column else "{:.3f}" for column in practitioner_table.columns},
    title="For comparison: practitioner IR = active return / tracking error (annualized)",
)
```

Sharpe ratios (annualized)

	Mean excess (%/yr)	Volatility (%/yr)	Sharpe ratio
TRBCX	9.85	17.14	0.575
GLCGX	8.53	16.35	0.522
STSEX	7.61	13.27	0.574
DFSVX	10.71	21.47	0.499
Market	9.31	15.27	0.610

Information ratio IR = α / σ(e) from the CAPM regression (annualized)

	Alpha (%/yr)	Beta	σ(e) (%/yr)	Information ratio
TRBCX	0.06	1.051	6.01	0.010
GLCGX	−1.05	1.029	4.49	−0.234
STSEX	0.14	0.803	5.06	0.027
DFSVX	−0.60	1.215	10.81	−0.056

For comparison: practitioner IR = active return / tracking error (annualized)

	Active return (%/yr)	Tracking error (%/yr)	Practitioner IR
TRBCX	0.54	6.05	0.089
GLCGX	−0.78	4.50	−0.173
STSEX	−1.70	5.88	−0.288
DFSVX	1.40	11.28	0.124

Interpretation (Part 2)

> **Answer.** Over 2002–2026, all four funds have a lower Sharpe ratio than the market (0.50–0.58 vs. 0.61). Their information ratios relative to the market are small under both definitions (α/σ(e): −0.23 to +0.03; practitioner IR: −0.29 to +0.12), so none of the funds delivered a meaningful risk-adjusted outperformance of the market.

Sharpe ratios. No fund has a higher Sharpe ratio than the market, although the differences are small. The market's Sharpe ratio is 0.61. The funds range from 0.50 (DFSVX) to 0.58 (TRBCX). DFSVX has the highest average excess return (10.7%/yr) but also by far the highest volatility (21.5%/yr), so it has the *lowest* Sharpe ratio.

Information ratios (IR = α / σ(e)). All four are small, between −0.23 and +0.03. For comparison, an IR of about 0.5 is usually considered good.

- **STSEX (+0.03)** and **TRBCX (+0.01)** have CAPM alphas of only +0.14% and +0.06% per year.
- **DFSVX (−0.06)** has an alpha of −0.60%/yr and the largest residual risk (σ(e) = 10.8%/yr).
- **GLCGX (−0.23)** has the most negative IR: an alpha of −1.05%/yr against little residual risk (σ(e) = 4.5%/yr). With about 24.6 years of data this corresponds to a t-statistic of only about −1.2 (t ≈ IR·√T), so it is not statistically significant.

What this means in the Treynor–Black model. By $S^* = \sqrt{S_M^2 + IR^2}$, the best positive-IR fund (STSEX) would raise the market's Sharpe ratio of 0.610 by less than 0.001. In theory, Treynor–Black would *short* GLCGX because of its negative alpha, which would raise S^* to about 0.65. But this relies on an in-sample alpha that is not statistically significant, and mutual fund shares cannot be sold short. Relative to the CAPM benchmark, none of the four funds therefore adds

meaningfully to a market index fund.

Why the two IRs differ. The practitioner IR does not adjust for β. The active return splits exactly into $\bar{r}_i - \bar{r}_M = \alpha_i + (\beta_i - 1)\,\bar{r}^e_M$:

- **DFSVX** (β ≈ 1.2) gets a *positive* practitioner IR (+0.12) because it beat the market by 1.4%/yr. That outperformance is compensation for extra market risk, and its α is negative.
- **STSEX** (β ≈ 0.8) gets a *negative* practitioner IR (−0.29) because it takes less market risk, although its α is slightly positive.

For two of the four funds, the sign depends on the definition. The two measures answer different questions. The practitioner IR shows whether investors in the fund did better than simply holding the index, including the effect of the fund's market risk. $\alpha/\sigma(e)$ shows the return the manager added beyond the fund's market exposure. We report both and use $\alpha/\sigma(e)$, the course definition, as our main measure.

A strong period for the market. From 2002 to 2026 the market earned 9.3% per year above the risk-free rate. Because this premium was large and positive, the β part of the practitioner IR favors high-β funds (DFSVX) and penalizes low-β funds (STSEX); in a weak market it would work the other way. The α-based IR is less sensitive to this, because it removes the market's contribution.

Part 3 checks whether the result still holds once we also control for size, value and momentum.

3. Is there evidence of manager skill? Factor regressions

We regress each fund's monthly excess return on the factors:

Fama–French 3-factor (FF3):

$$r^e_{i,t} = \alpha_i + \beta_i\,(r_M - r_f)_t + s_i\,SMB_t + h_i\,HML_t + e_{i,t}$$

Fama–French–Carhart 4-factor (FFC4):

$$r^e_{i,t} = \alpha_i + \beta_i\,(r_M - r_f)_t + s_i\,SMB_t + h_i\,HML_t + m_i\,MOM_t + e_{i,t}$$

The factor loadings capture the return the fund earns from its *style* (market, size, value, momentum), which an investor could get cheaply from passive factor portfolios. **Skill shows up in the intercept α.** It is the average return the factors cannot explain.

- **Test:** $H_0 : \alpha_i = 0$ vs. $H_1 : \alpha_i \neq 0$. Evidence of skill requires $\alpha > 0$ **and** statistical significance ($|t| > 1.96$ at the 5% level).
- **Standard errors:** Newey–West (HAC), robust to heteroskedasticity and autocorrelation. The lag length follows the rule of thumb $\lfloor 4(T/100)^{2/9} \rfloor$. Plain OLS t-statistics are shown for comparison.
- Significance stars: * 10%, ** 5%, *** 1%.

```
MODELS = {
    "FF3": ["Mkt-RF", "SMB", "HML"],
    "FFC4": ["Mkt-RF", "SMB", "HML", "Mom"],
}
T = len(data)
NW_LAGS = int(np.floor(4 * (T / 100) ** (2 / 9)))
print(f"T = {T} months, Newey-West lags = {NW_LAGS}")


def fit_factor_model(fund, factor_cols, hac=True):
    X = sm.add_constant(data[factor_cols])
    model = sm.OLS(excess[fund], X)
    if hac:
        return model.fit(cov_type="HAC", cov_kwds={"maxlags": NW_LAGS})
    return model.fit()


fits = {
    (model, fund): fit_factor_model(fund, cols)
    for model, cols in MODELS.items()
    for fund in TICKERS
}
ols_fits = {
    (model, fund): fit_factor_model(fund, cols, hac=False)
    for model, cols in MODELS.items()
    for fund in TICKERS
}
```

```
T = 295 months, Newey-West lags = 5
```

```
LABELS = {
    "const": "α (% per month)",
    "Mkt-RF": "β (MKT)",
    "SMB": "s (SMB)",
    "HML": "h (HML)",
```

```
    "Mom": "m (MOM)",
}


def stars(p):
    return "***" if p < 0.01 else "**" if p < 0.05 else "*" if p < 0.10 else ""


def regression_table(model):
    # Coefficients with significance stars, Newey-West t-statistics in parentheses below.
    terms = ["const", *MODELS[model]]
    index = [label for term in terms for label in (LABELS[term], "")] + ["adj. R²", "N"]
    columns = {}
    for fund in TICKERS:
        res = fits[(model, fund)]
        cells = []
        for term in terms:
            scale = 100 if term == "const" else 1  # alpha in % per month
            coef = f"{res.params[term] * scale:.3f}{stars(res.pvalues[term])}"
            cells += [coef, f"({res.tvalues[term]:.2f})"]
        cells += [f"{res.rsquared_adj:.3f}", f"{int(res.nobs)}"]
        columns[fund] = cells
    return pd.DataFrame(columns, index=index)


show_table(
    regression_table("FF3"),
    title="Fama-French 3-factor model (Newey-West t-statistics in parentheses)",
)
```

Fama-French 3-factor model (Newey-West t-statistics in parentheses)

	TRBCX	GLCGX	STSEX	DFSVX
α (% per month)	0.003	−0.090	−0.011	0.005
	(0.04)	(−1.61)	(−0.18)	(0.09)
β (MKT)	1.090***	1.058***	0.847***	1.029***
	(57.20)	(69.81)	(38.01)	(62.87)
s (SMB)	−0.081***	−0.068**	−0.289***	0.848***
	(−2.92)	(−2.51)	(−8.44)	(23.92)
h (HML)	−0.395***	−0.272***	0.100***	0.636***
	(−14.69)	(−11.68)	(3.71)	(26.32)
adj. R^2	0.940	0.958	0.890	0.974
N	295	295	295	295

```
show_table(
    regression_table("FFC4"),
    title="Fama-French-Carhart 4-factor model (Newey-West t-statistics in parentheses)",
)
```

Fama-French-Carhart 4-factor model (Newey-West t-statistics in parentheses)

	TRBCX	GLCGX	STSEX	DFSVX
α (% per month)	−0.003	−0.106*	0.010	0.012
	(−0.05)	(−1.87)	(0.16)	(0.21)
β (MKT)	1.095***	1.071***	0.832***	1.024***
	(53.55)	(77.80)	(29.78)	(54.91)
s (SMB)	−0.080***	−0.066**	−0.291***	0.847***
	(−2.95)	(−2.42)	(−8.28)	(23.45)
h (HML)	−0.391***	−0.261***	0.085***	0.632***
	(−12.59)	(−10.37)	(3.09)	(26.26)
m (MOM)	0.014	0.035*	−0.044	−0.014
	(0.56)	(1.89)	(−1.34)	(−0.77)
adj. R^2	0.940	0.959	0.892	0.974
N	295	295	295	295

Alpha summary: FF3 vs. FFC4

```
rows = {}
for fund in TICKERS:
    row = {}
    for model in MODELS:
        nw, ols = fits[(model, fund)], ols_fits[(model, fund)]
        row[f"{model} α (%/yr)"] = nw.params["const"] * ANN * 100
        row[f"{model} t (NW)"] = nw.tvalues["const"]
        row[f"{model} t (OLS)"] = ols.tvalues["const"]
        row[f"{model} p (NW)"] = nw.pvalues["const"]
    rows[fund] = row
alpha_summary = pd.DataFrame.from_dict(rows, orient="index")

show_table(
    alpha_summary,
    "{:.2f}",
    title="Annualized alphas with Newey-West and plain OLS t-statistics",
)
```

Annualized alphas with Newey-West and plain OLS t-statistics

	FF3 α (%/yr)	FF3 t (NW)	FF3 t (OLS)	FF3 p (NW)	FFC4 α (%/yr)	FFC4 t (NW)	FFC4 t (OLS)	FFC4 p (NW)
TRBCX	0.04	0.04	0.05	0.96	−0.04	−0.05	−0.05	0.96
GLCGX	−1.08	−1.61	−1.57	0.11	−1.28	−1.87	−1.86	0.06
STSEX	−0.13	−0.18	−0.14	0.86	0.12	0.16	0.13	0.87
DFSVX	0.07	0.09	0.09	0.92	0.14	0.21	0.20	0.84

Interpretation (Part 3)

FF3 model: no evidence of skill. None of the four alphas is statistically different from zero:

Fund	α (%/yr)	t (NW)	Verdict
TRBCX	+0.04	0.04	essentially zero
GLCGX	−1.08	−1.61	negative, not significant (p = 0.11)
STSEX	−0.13	−0.18	essentially zero
DFSVX	+0.07	0.09	essentially zero

FFC4 model: same conclusion. Adding momentum barely changes anything. Still no fund has a significantly **positive** alpha:

Fund	α (%/yr)	t (NW)	Verdict
TRBCX	−0.04	−0.05	essentially zero
GLCGX	−1.28	−1.87	negative, significant at 10% only (p = 0.06)
STSEX	+0.12	0.16	essentially zero
DFSVX	+0.14	0.21	essentially zero

Compared with FF3, the alphas move by at most 0.25 %-points per year, and the adjusted R^2 values are essentially unchanged. The only change is that **GLCGX's** alpha becomes slightly more negative and is now significant at the 10% level. GLCGX has a small positive momentum loading (m = 0.035, significant at 10%). Part of its return is therefore explained by momentum exposure, which leaves even less for the manager. This is weak evidence of **underperformance**, not skill.

The factor loadings match the fund styles:

- **TRBCX and GLCGX** (large-cap growth): β slightly above 1, negative SMB (large firms), strongly negative HML (growth stocks).
- **STSEX** (large-cap blend): $\beta \approx 0.85$, strongly negative SMB (mega-caps), small positive HML.
- **DFSVX** (small-cap value): large positive SMB (0.85) and HML (0.64), exactly as the name says.
- Momentum loadings are small and not significant at 5% for any fund. That's why FF3 and FFC4 give almost the same alphas.

The factors explain almost everything. The adjusted R^2 is 0.89–0.97. These funds' returns are mostly their factor exposures, which an investor could replicate cheaply with passive factor/index portfolios. DFSVX ($R^2 = 0.974$) is essentially a systematic size-and-value tilt. Its outperformance of the market comes from its SMB/HML exposure, not from stock picking: its alpha moves from −0.60%/yr in the CAPM to about zero (+0.07%/yr) once size and value are controlled for. Delivering a factor exposure closely and at low cost can itself be seen as a kind of skill, but it is not stock-picking alpha.

Conclusion. Neither the FF3 nor the FFC4 model shows evidence of manager skill (a significantly positive α) for any of the four funds over 2002–2026. Two caveats:

- Fund returns on Yahoo are **net of fees**. An alpha of about zero after fees means that, at best, the managers generate just enough gross alpha to cover their fees, so investors capture none of it. This matches Fama & French (2010) for the average active fund: gross α close to zero, net α negative. Berk & Green (2004) show that a zero net α is still compatible with skilled managers, because fund inflows compete the alpha away.
- **Survivorship bias:** all four funds still exist in 2026, so they were selected after the fact. This favors finding positive alphas, which makes the absence of skill even more telling.